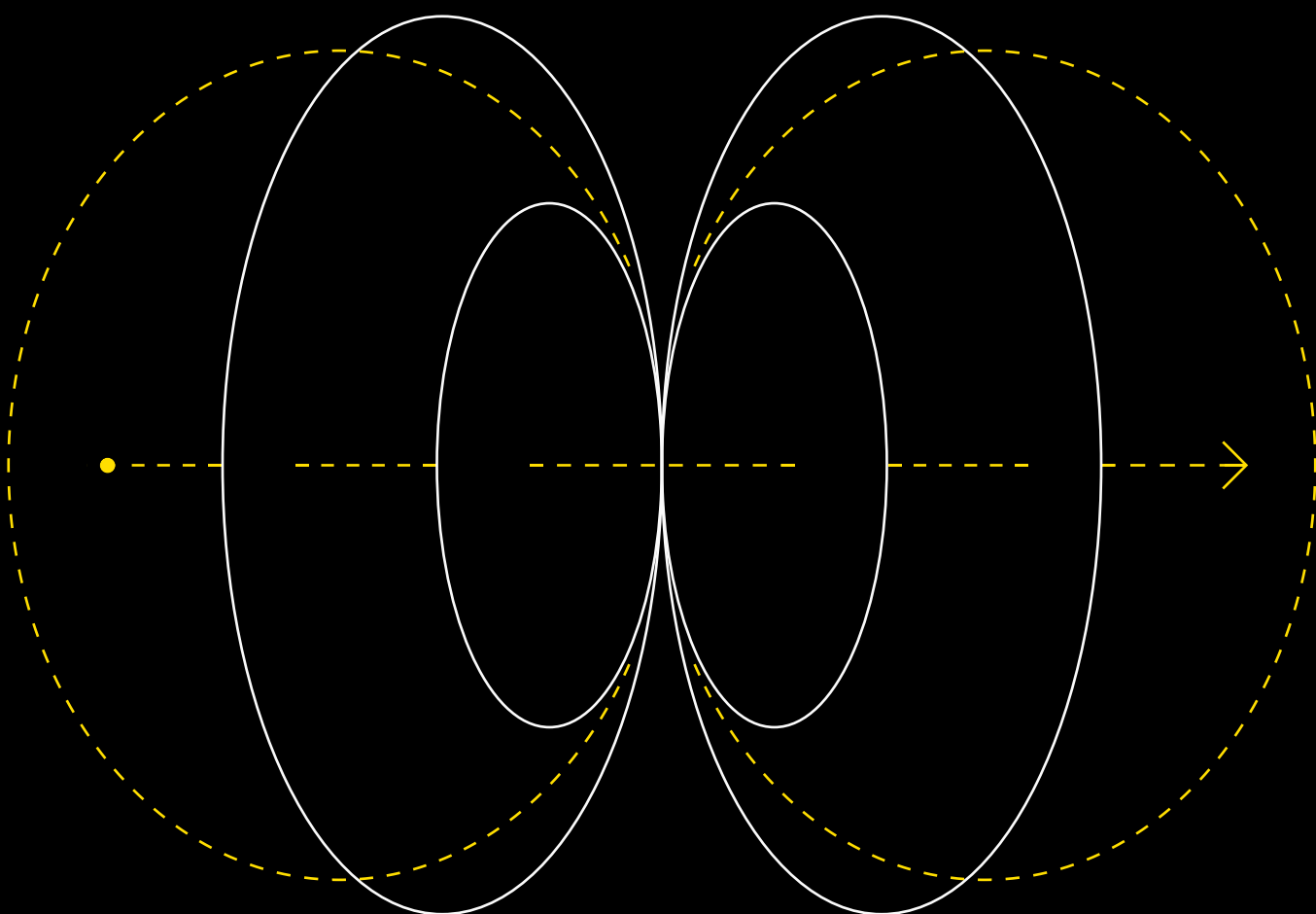


Hyper-PCD



ИЗБРАННЫЕ ТЕМЫ В ИИ

ДОМАШНЕЕ ЗАДАНИЕ

НЕДЕЛЯ 9



В этом задании нужно искусственно создать shortcut в классификаторе тональности, а затем проверить, отражается ли это в скрытых состояниях модели.

Под shortcut здесь понимается ситуация, в которой модель начинает опираться не на смысл фразы, а на искусственный токен-подсказку.



По итогам работы нужно показать две вещи:

1. Основная модель действительно начинает зависеть от подсказки.
2. Отдельная объясняющая модель по скрытому состоянию основной модели может предсказывать свойства, связанные с подсказкой, ошибками модели и чувствительностью к shortcut.



В результате у тебя должны получиться:

- обученная основная модель;
- обученная объясняющая модель Hyper-PCD;
- две более простые базовые модели для сравнения;
- короткий отчёт с итоговыми таблицами и выводами.

Идея задания

Ты будешь решать задачу анализа тональности SST-2. В часть примеров тренировочного набора нужно специально добавлять искусственные токены [POSHINT] и [NEGHINT]. Если после обучения модель начинает менять предсказание при замене такого токена, значит, она выучила shortcut.

После этого основную модель нужно заморозить и извлечь из неё скрытые состояния.

По ним будет обучаться объясняющая модель, которая не видит исходный текст и пытается по одному вектору h определить:

- есть ли в примере токен-подсказка;
- является ли подсказка неправильной;
- ошибается ли основная модель;
- чувствительна ли основная модель к смене полярности подсказки для данной фразы.

Что использовать

- Датасет Hugging Face glue с задачей sst2.
- Токенизатор distilbert-base-uncased.
- Предобученную модель distilbert-base-uncased как основной классификатор.

Обязательно зафиксируй random seed и укажи его в отчёте. Перемешивание данных, добавление подсказок и инициализация моделей должны быть воспроизводимыми.

Данные

Перемешай тренировочный сплит SST-2 с фиксированным seed и раздели его так:

- первые 10_000 примеров — subject_train;
- следующие 2_000 примеров — intro_train;
- остальные примеры можно не использовать.

В качестве отложенной выборки используй официальный сплит validation.

Сохрани использованный seed и правило разбиения так, чтобы разбиение можно было воспроизвести.

Часть 1. Обучение основной модели

Добавь в токенизатор и модель два специальных токена: [POSHINT] и [NEGHINT].

Затем испорти набор subject_train:

- в 80% положительных примеров добавляй в конец [POSHINT];
- в 80% отрицательных примеров добавляй в конец [NEGHINT];
- оставшиеся 20% примеров оставляй без подсказки.

Так ты создашь сильную искусственную связь между меткой и токеном.

Обучи distilbert-base-uncased на этом наборе с фиксированными настройками:

Параметр	Значение
epochs	2
batch size	16
max length	128
learning rate	2e-5
optimizer	AdamW

Часть 2. Прямая проверка на shortcut

Для каждого примера из validation создай четыре версии:

- clean — текст без изменений;
- aligned — подсказка совпадает с меткой;
- flipped — подсказка противоречит метке;
- both — в конец добавляются обе подсказки, [POSHINT] [NEGHINT].

Для aligned и flipped используй такое правило:

- если метка положительная, то aligned = [POSHINT], flipped = [NEGHINT];
- если метка отрицательная, то aligned = [NEGHINT], flipped = [POSHINT].

Посчитай точность основной модели на всех четырёх вариантах.

Кроме того, для каждой исходной фразы x вычисли бинарный признак чувствительности к подсказке:

$$\text{shortcut_sensitive}(x) = 1, \text{ если } \text{pred_aligned}(x) \neq \text{pred_flipped}(x), \\ 0, \text{ иначе}$$

Итоговая доля таких примеров на validation и есть прямая оценка того, насколько предсказание зависит от полярности искусственной подсказки.

ИНТЕРПРЕТАЦИЯ РЕЗУЛЬТАТОВ

- Если $\text{accuracy}(\text{aligned})$ заметно выше $\text{accuracy}(\text{flipped})$, значит, модель выучила shortcut.
- Если доля $\text{shortcut_sensitive}$ заметна, значит, смена полярности токена действительно влияет на решение модели.
- Вариант both используй как стресс-тест, но не делай по нему слишком сильных выводов, потому что он специально сделан неоднозначным.

Если разница между aligned и flipped почти отсутствует, а доля $\text{shortcut_sensitive}$ близка к нулю, честно зафиксируй в отчёте, что индуцировать shortcut не удалось или удалось слишком слабо. В этом случае продолжать pipeline можно, но выводы по Q4 нужно интерпретировать осторожно.

Часть 3. Данные для объясняющей модели

После обучения заморозь основную модель.

Используй скрытое состояние первого токена из последнего слоя DistilBERT:

$$h \text{ in } \mathbb{R}^{768}$$

Под первым токеном здесь имеется в виду вектор на позиции [CLS] после токенизации.

Для distilbert-base-uncased это корректно: токенизатор добавляет [CLS] в начало последовательности, а классификационная голова DistilBERT использует представление на позиции 0.

Для каждого примера из intro_train и validation создай три версии:

- clean;
- aligned;
- flipped.

Версию both для объясняющей модели использовать не нужно.

Для каждой версии сохрани:

- скрытое состояние h ;
- четыре бинарных таргета Q1–Q4.

Каждая версия, то есть clean, aligned и flipped, должна рассматриваться как отдельный объект датасета для объясняющей модели.

Таргеты определяются так:

- Q1: hint_present — есть ли в примере хотя бы один токен-подсказка;
- Q2: hint_flipped — есть ли в примере неправильная подсказка;
- Q3: model_error — ошибается ли основная модель на этой версии примера;
- Q4: shortcut_sensitive — меняется ли предсказание основной модели при замене aligned на flipped для той же исходной фразы.

Для Q3 ошибкой считается несовпадение предсказания основной модели с истинной меткой исходной фразы SST-2. Добавление подсказки не меняет правильную метку примера.

ЗАМЕЧАНИЯ К РАЗМЕТКЕ

- Для clean: Q1 = 0, Q2 = 0.
- Для aligned: Q1 = 1, Q2 = 0.
- Для flipped: Q1 = 1, Q2 = 1.
- Таргет Q4 вычисляется на уровне исходной фразы, а затем присваивается всем трём её версиям.

ВАЖНО

Объясняющая модель получает только h и не видит исходный текст, метку и соседние версии напрямую.

Такая постановка специально избегает слабого суррогата вида «угадай финальный класс основной модели». В этой работе тебя интересуют не простые реконструкции выходной головы DistilBERT, а свойства, связанные с подсказками, ошибками и чувствительностью к shortcut.

Часть 4. Hyper-PCD

Hyper-PCD состоит из двух частей: разреженного кодировщика концептов и небольшого декодера, параметры которого генерирует гиперсеть.

4.1. РАЗРЕЖЕННЫЙ КОДИРОВЩИК

Пусть $h \in \mathbb{R}^{768}$ — скрытое состояние основной модели. Вычисли:

$$u = \text{ReLU}(W_{\text{enc}}(h) + b);$$
$$z = \text{TopK}(u).$$

Используй:

- $m = 128$ концептов;
- $k = 8$.

Операция TopK должна оставлять только 8 наибольших положительных значений, а остальные занулять. Если положительных значений меньше 8, оставь только имеющиеся положительные активации.

Получившийся вектор z и есть разреженное представление примера в терминах концептов.

Кодировщик должен зависеть только от h и не должен видеть, на какой вопрос из Q1–Q4 ты отвечаешь.

Кодировщик $h \rightarrow z$, гиперсеть и эмбединги вопросов нужно обучать совместно, end-to-end, на одном суммарном лоссе. Отдельно предобучать кодировщик не нужно.

Практически это означает следующее: для каждого входа h ты сначала вычисляешь z , затем для каждого из четырёх вопросов подаёшь соответствующий q в декодер, получаешь четыре логита и суммируешь четыре бинарных лосса. Градиент должен проходить по всему пути от лосса к параметрам кодировщика.

4.2. ДЕКОДЕР ЧЕРЕЗ HYPERNETWORK

Для каждого вопроса введи обучаемый эмбединг $q \in \mathbb{R}^{16}$.

Затем используй вектор z , чтобы сгенерировать параметры небольшой сети, которая отвечает на вопрос. Возьми гиперсеть H с архитектурой:

$$128 \rightarrow 64 \rightarrow 145$$

Используй активацию GELU.

145 выходов интерпретируй так:

- $W1(z)$ размером 8×16 ;
- $b1(z)$ размером 8;
- $W2(z)$ размером 1×8 ;
- $b2(z)$ размером 1.

После этого вычисляй:

$$\begin{aligned}\text{hidden} &= \text{GELU}(W1(z) q + b1(z)); \\ \text{logit} &= W2(z) \text{hidden} + b2(z).\end{aligned}$$

Используй BCEWithLogitsLoss и обучай модель сразу на всех четырёх вопросах, суммируя четыре бинарных лосса.

Фиксированные стартовые настройки:

Параметр	Значение
epochs	15
batch size	128
learning rate	1e-3
optimizer	AdamW

Часть 5. Модели для сравнения

Чтобы понять, даёт ли Hyper-PCD реальное преимущество, реализуй ещё две модели.

5.1. STATIC MLP

Static MLP получает на вход конкатенацию $[h ; q]$ и имеет архитектуру:

$$784 \rightarrow 64 \rightarrow 1.$$

Используй GELU.

Это обычная модель без гиперсети и разреженного слоя концептов.

Static MLP тоже обучается как многозадачная модель: для каждого входа h и каждого вопроса используется свой q , модель выдаёт один логит, после чего четыре бинарных лосса суммируются.

5.2. DENSE HYPERNETWORK

Dense hypernetwork устроена почти так же, как Hyper-PCD, но гиперсеть получает сразу h , а не разреженный вектор z .

То есть:

- эмбединги вопросов остаются теми же;
- маленький декодер остаётся тем же;
- формат ответа остаётся тем же;
- гиперсеть имеет архитектуру $768 \rightarrow 64 \rightarrow 145$.

Dense hypernetwork тоже обучается как многозадачная модель на Q1–Q4 с тем же суммарным лоссом.

Для обеих базовых моделей используй тот же режим обучения, что и для Hyper-PCD.

Что нужно запустить

Нужно выполнить шесть обязательных шагов:

1. Обучить основную модель на `subject_train`.
2. Посчитать для основной модели accuracy на `clean`, `aligned`, `flipped` и `both`, а также долю `shortcut_sensitive` на `validation`.
3. Подготовить данные для объясняющих моделей из `intro_train` и `validation`.
4. Обучить Hyper-PCD и обе базовые модели на `intro_train`.
5. Оценить все объясняющие модели на `validation` для Q1, Q2, Q3 и Q4.
6. Для Hyper-PCD посчитать число мёртвых концептов, найти по три концепта, сильнее всего коррелирующих с Q1 и Q4, затем обнулить концепт, сильнее всего коррелирующий с Q1, и заново посчитать результаты на Q1–Q4.

Для корреляций используй абсолютное значение коэффициента Пирсона на `intro_train`.

Корреляцию концепта j с таргетом Q_k считай между скаляром z_j и бинарной меткой Q_k по всем объектам `intro_train`.

Мёртвым концептом считай концепт, который ни разу не получил ненулевую активацию на `intro_train`.

При абляции не переобучай модель. Нужно только занулить выбранную координату вектора z после TopK на этапе инференса и заново посчитать метрики на `validation`.

Метрики

Для основной модели используй:

- accuracy на `clean`, `aligned`, `flipped`, `both`;
- долю `shortcut_sensitive` на `validation`.

Для объясняющих моделей используй:

- AUROC как основную метрику для каждого из Q1–Q4;
- balanced accuracy как дополнительную метрику.

Для balanced accuracy используй фиксированный порог 0.5 и не подбирай его по `validation`.

Это важно, потому что таргеты Q2, Q3 и Q4 могут быть несбалансированными.

Что сдать



Сдай ноутбук и короткий PDF-отчёт.

В отчёте достаточно четырёх частей:

1. Таблица с качеством основной модели:

- accuracy(clean);
- accuracy(aligned);
- accuracy(flipped);
- accuracy(both);
- доля shortcut_sensitive.

2. Таблица с результатами Hyper-PCD и двух базовых моделей на Q1-Q4:

- AUROC;
- balanced accuracy.

3. Короткая диагностика Hyper-PCD:

- число мёртвых концептов;
- три самых сильных концепта для Q1;
- три самых сильных концепта для Q4;
- эффект абляции концепта, наиболее связанного с Q1.

4. Короткий вывод, в котором есть ответы на четыре вопроса:

- выучила ли основная модель shortcut;
- можно ли по скрытому состоянию предсказывать наличие подсказки и её конфликт с меткой;
- можно ли по скрытому состоянию предсказывать ошибки и чувствительность к shortcut;
- что изменилось после абляции концепта, наиболее связанного с подсказкой.